Kubernetes Interview Guide

0–3 Years Experience | AWS DevOps Interview Preparation | 25 Core Questions

1. What is Kubernetes?

- Kubernetes (K8s) is an open-source **container orchestration platform** originally developed by Google and now maintained by the CNCF.
- It automates the **deployment, scaling, load balancing, and self-healing** of containerised applications.
- Kubernetes abstracts the underlying infrastructure so you declare *what* you want (desired state) and it continuously works to maintain that state.
- Core components: API Server, etcd, Scheduler, Controller Manager (control plane) and kubelet, kube-proxy, container runtime (worker nodes).

2. Why do companies use Kubernetes?

- **Auto-scaling:** Scales pods up/down based on CPU, memory, or custom metrics via HPA/VPA.
- **High availability:** Automatically restarts failed containers and reschedules them on healthy nodes.
- **Efficient resource utilisation:** Bin-packs containers across nodes, reducing infrastructure costs.
- **Declarative configuration:** YAML manifests act as version-controlled infrastructure definitions.
- **Portability:** Runs identically on any cloud (AWS EKS, GKE, AKS) or on-premise — avoids vendor lock-in.
- **Rolling updates & rollbacks:** Zero-downtime deployments with instant rollback capability.

3. What problem does Kubernetes solve?

- **Manual container management at scale:** Running hundreds of Docker containers manually across many servers is error-prone.
- **Service discovery:** Containers get dynamic IPs; Kubernetes provides stable DNS-based discovery.
- **Load balancing:** Distributes traffic automatically across healthy pod replicas.
- **Self-healing:** Detects and replaces crashed containers without manual intervention.
- **Configuration sprawl:** Centralises config via ConfigMaps and Secrets instead of baking values into images.
- **Scheduling complexity:** Automatically places workloads on nodes that have sufficient CPU/memory.

4. What is a Kubernetes Cluster?

- A Kubernetes cluster is a set of machines (nodes) that together run containerised workloads managed by Kubernetes.
- It consists of a **Control Plane** (manages the desired state) and one or more **Worker Nodes** (run actual workloads).
- The cluster is the top-level unit of isolation — each cluster has its own API server, etcd store, and network.
- In managed services like AWS EKS, the control plane is fully managed by the cloud provider.

5. What is a Node?

- A Node is a single physical or virtual machine that is part of the Kubernetes cluster.
- Each node runs the **kubelet** (agent communicating with the API server), **kube-proxy** (network rules), and a container runtime (containerd/CRI-O).
- Nodes can be **Master Nodes** (control plane) or **Worker Nodes** (data plane).
- Node capacity (CPU, RAM) is tracked by the scheduler when placing pods.

6. What is a Master Node?

- The Master Node (Control Plane) hosts components that make global cluster decisions.
- **kube-apiserver:** Single entry point for all REST operations; validates and processes requests.
- **etcd:** Distributed key-value store holding all cluster state and configuration.
- **kube-scheduler:** Watches for unscheduled pods and assigns them to suitable nodes.
- **kube-controller-manager:** Runs controllers (ReplicaSet, Node, Job, etc.) to maintain desired state.
- In production, the control plane runs in HA mode (multiple master nodes) to avoid single point of failure.

7. What is a Worker Node?

- Worker Nodes are the machines that actually **run application pods**.
- **kubelet:** Agent that receives pod specs from the API server and manages container lifecycle.
- **kube-proxy:** Maintains iptables/IPVS rules for pod-to-service networking.
- **Container runtime:** (containerd, CRI-O) pulls images and runs containers.
- Worker nodes report their status and resource availability back to the control plane continuously.

8. What is a Pod?

- A Pod is the **smallest deployable unit** in Kubernetes — a wrapper around one or more tightly-coupled containers.
- All containers in a pod share the same **network namespace** (IP address, ports) and can share **storage volumes**.
- Pods are ephemeral — they can be created, destroyed, and replaced at any time.
- Common pattern: one main container + sidecar containers (e.g., log shipper, service mesh proxy).
- Pod spec defines: image, resource requests/limits, environment variables, volume mounts, health probes.

9. Why is Pod the smallest deployable unit?

- Kubernetes schedules and manages resources at the **pod level**, not the individual container level.
- Containers inside a pod share the same Linux namespaces (network, IPC, UTS), making them tightly coupled by design.
- This co-location model mirrors how you'd run helper processes (sidecars, init containers) alongside a main process on a single host.
- Kubernetes cannot schedule individual containers independently — the pod is the atomic unit of placement and scaling.
- Direct analogy: a pod is like a logical host; containers inside it are like processes on that host.

10. What is a Deployment?

- A Deployment is a **higher-level abstraction** that manages a ReplicaSet and, by extension, a set of identical pods.
- It defines the **desired state**: which image to use, how many replicas, update strategy, and resource limits.
- Supports **rolling updates** (gradually replaces old pods) and **rollback** (reverts to a previous ReplicaSet).
- Key fields: *replicas*, *selector*, *template*, *strategy* (RollingUpdate or Recreate).
- In practice, you almost always create a Deployment rather than a bare pod or ReplicaSet.

11. What is a ReplicaSet?

- A ReplicaSet ensures a **specified number of identical pod replicas** are running at all times.
- If a pod crashes or is deleted, the ReplicaSet controller creates a replacement automatically.
- It uses a **label selector** to identify which pods it owns.
- ReplicaSets are generally **not created directly** — Deployments manage them. Each rolling update creates a new ReplicaSet.
- Old ReplicaSets are retained (scaled to 0) to enable rollback.

12. What is a Service?

- A Service provides a **stable network endpoint** (DNS name + ClusterIP) for accessing a dynamic set of pods.
- Pods have ephemeral IPs that change on restart; Services abstract this with a **consistent virtual IP**.
- Services use **label selectors** to route traffic to matching pods — no tight coupling to pod identity.
- kube-proxy maintains iptables/IPVS rules on each node to implement the virtual IP routing.

13. What are the different types of Services?

- **ClusterIP** (default): Exposes the service on an internal cluster IP — reachable only within the cluster.
- **NodePort**: Exposes a static port (30000–32767) on every node's IP — accessible from outside the cluster.
- **LoadBalancer**: Provisions a cloud load balancer (AWS ELB, GCP LB) and assigns an external IP.
- **ExternalName**: Maps the service to a DNS name (e.g., an external database) — returns a CNAME, no proxying.
- **Headless** (ClusterIP: None): Returns individual pod IPs via DNS — used for StatefulSets and direct pod discovery.

14. What is ClusterIP?

- ClusterIP is the **default Service type** — assigns a virtual IP accessible only within the cluster.
- Ideal for **internal microservice communication** — e.g., frontend pod calling backend pod.
- DNS format: <service-name>.<namespace>.svc.cluster.local
- Not accessible from outside the cluster without an additional Ingress or NodePort.

15. What is NodePort?

- NodePort exposes a service on a **static port on every node** in the cluster (range: 30000–32767).
- External traffic can reach the service at **<NodeIP>:<NodePort>**.
- Kubernetes automatically creates a ClusterIP service and routes NodePort traffic to it.
- **Limitation:** Exposes a port on every node — not suitable for production without a load balancer in front.
- Use case: Quick external access during development/testing without a cloud load balancer.

16. What is LoadBalancer Service?

- Extends NodePort by additionally **provisioning a cloud provider load balancer** (e.g., AWS ELB/NLB).
- Automatically assigns an **external IP or DNS hostname** for outside access.
- Internally, it still creates a ClusterIP and NodePort — the cloud LB routes to the NodePort.
- **Cost consideration:** Each LoadBalancer service provisions a separate cloud LB, which can be expensive at scale.
- For multiple services, use an **Ingress Controller** with a single LoadBalancer instead.

17. What is a Namespace?

- A Namespace provides **virtual isolation** within a single Kubernetes cluster — like a folder inside the cluster.
- Allows multiple teams/environments to share one cluster without interfering with each other.
- Resources (pods, services, configmaps) are **namespaced**; some (nodes, PersistentVolumes) are cluster-scoped.
- Default namespaces: *default*, *kube-system*, *kube-public*, *kube-node-lease*.
- RBAC policies and NetworkPolicies can be scoped to namespaces for security isolation.

18. What is ConfigMap?

- A ConfigMap stores **non-sensitive configuration data** as key-value pairs, decoupling config from container images.
- Can be consumed by pods as: **environment variables**, **command-line arguments**, or **mounted files** in a volume.
- Allows the same Docker image to run in dev, staging, and production with different configs.
- **Limitation:** ConfigMaps are not encrypted — never store passwords or API keys here (use Secrets instead).
- Max size: 1 MiB per ConfigMap.

19. What is a Secret?

- Secrets store **sensitive data** (passwords, tokens, TLS certs) separately from pod specs.
- Values are **base64-encoded** at rest in etcd — note: this is encoding, NOT encryption by default.
- For true security, enable **etcd encryption at rest** and use external secrets managers (AWS Secrets Manager via External Secrets Operator, HashiCorp Vault).
- Types: *Opaque* (generic), *kubernetes.io/tls*, *kubernetes.io/dockerconfigjson*, *kubernetes.io/service-account-token*.
- Best practice: avoid committing Secrets to Git — use Sealed Secrets or External Secrets Operator.

20. What is Ingress?

- Ingress is an API object that manages **external HTTP/HTTPS routing** into cluster services — acting as a Layer 7 load balancer.
- Requires an **Ingress Controller** to be installed (NGINX, AWS ALB Ingress Controller, Traefik, etc.).
- Supports: **path-based routing** (/api → backend, /app → frontend), **host-based routing** (api.example.com), TLS termination.
- Single Ingress can replace multiple LoadBalancer services — significantly reducing cloud LB costs.
- Annotations on the Ingress resource configure controller-specific behaviour (e.g., SSL redirect, rate limiting).

21. What is Persistent Volume (PV)?

- A PersistentVolume is a piece of **cluster-level storage** provisioned by an admin or dynamically via a StorageClass.
- It has a **lifecycle independent of any pod** — data persists even if the pod is deleted.
- Backed by: AWS EBS, EFS, GCE PD, NFS, local disk, etc.
- Key attributes: *capacity*, *accessModes* (ReadWriteOnce, ReadOnlyMany, ReadWriteMany), *reclaimPolicy* (Retain, Delete, Recycle).

22. What is Persistent Volume Claim (PVC)?

- A PVC is a **user's request for storage** — similar to how a Pod requests CPU/memory, a PVC requests storage.
- Kubernetes **binds a PVC to a PV** that satisfies the requested size and access mode.
- Pods reference PVCs in their spec, not PVs directly — provides an abstraction layer.
- With **dynamic provisioning**, a StorageClass automatically creates a PV when a PVC is submitted.
- PVC status: *Pending* (awaiting binding), *Bound* (matched to a PV), *Lost* (PV deleted).

23. What is the difference between Docker and Kubernetes?

- **Docker** is a container runtime and toolset — it builds, runs, and manages containers on a *single host*.
- **Kubernetes** is a container orchestrator — it manages containers across a *cluster of many hosts*.
- Docker handles: image build, image registry, container start/stop on one machine.
- Kubernetes handles: scheduling across nodes, auto-scaling, self-healing, service discovery, rolling updates at scale.
- They are **complementary, not competing**: Kubernetes uses a container runtime (Docker/containerd) to run the actual containers.
- Analogy: Docker is the individual ship; Kubernetes is the entire shipping fleet management system.

24. Why is Kubernetes important in DevOps?

- **Enables CI/CD pipelines** — new images are automatically deployed via rolling updates with zero downtime.
- **Infrastructure as Code:** YAML manifests are version-controlled, reviewed, and audited like application code.
- **Environment parity:** Same manifests run in dev, staging, and production — reduces 'works on my machine' issues.
- **Observability:** Integrates with Prometheus, Grafana, Datadog for metrics, logs, and tracing.
- **GitOps workflows:** Tools like ArgoCD/Flux continuously reconcile cluster state with Git — full audit trail.
- **Multi-cloud strategy:** Kubernetes APIs are portable — reduces dependency on any single cloud provider.

25. What are the most commonly used Kubernetes resources?

- **Pod** — atomic unit of deployment (rarely created directly).
- **Deployment** — manages stateless application replicas and rolling updates.
- **StatefulSet** — manages stateful apps (databases) requiring stable identity and ordered deployment.
- **DaemonSet** — runs one pod per node (log agents, monitoring agents, CNI plugins).
- **Job / CronJob** — run-to-completion tasks and scheduled jobs.
- **Service** — stable network access to pods (ClusterIP, NodePort, LoadBalancer).
- **Ingress** — HTTP/HTTPS routing from outside the cluster.
- **ConfigMap / Secret** — externalised configuration and sensitive data.
- **PersistentVolumeClaim (PVC)** — requests durable storage for stateful workloads.
- **HorizontalPodAutoscaler (HPA)** — auto-scales pod count based on metrics.
- **RBAC (Role/ClusterRole/RoleBinding)** — fine-grained access control.
- **NetworkPolicy** — controls pod-to-pod traffic at the network layer.